

MAGE-Ontology: A Domain Ontology for Ghidra Malware Analysis

Emily Miller¹, Cogan Shimizu¹

¹Wright State University, 3640 Colonel Glenn Hwy, Dayton, Ohio 45435

Abstract

Software reverse engineering provides a strong foundation for effective malware analysis. Specialized tools such as Ghidra provide disassembly and decompilation capabilities for a given executable file. These analytical views reveal a program's structure and functionality, which helps mitigate malware when source code is unavailable. While these tools are useful, they often produce excessive data that can be challenging to interpret. This paper introduces the MAGE-Ontology: an ontology for malware analysis in Ghidra for executables. The goal is to organize data from Ghidra's executable analysis into a graphical framework to categorize malware. We leveraged the MAGE-ontology to construct knowledge graphs based on malware executables, which were able to detect different forms of malicious behavior utilizing SPARQL queries.

Keywords

Software Reverse Engineering, Ghidra, Executable, Disassembly, Decompilation

1. Introduction

The impact of malware has significantly evolved in the past decade. Not only have many forms of malware been developed, but their complexity has increased, causing significant financial and security damages to both industries and individuals. In 2023, the average data breach was calculated to cost around \$3.86 million, with that number increasing in the past few years [1]. One of the most common types of malware that leads to financial loss is ransomware, which uses encryption to block access to important data and demands a ransom to return the data through the decryption key [2]. Other common malware variants include Trojans, viruses, and worms. Trojans compromise systems by masquerading as legitimate downloadable software, whereas viruses rely on user interaction with malicious links or files, and worms propagate autonomously across networks [1].

Efforts have been made to analyze and understand malware due to its severe impact, with two main categories of malware analysis: dynamic and static [3]. Dynamic analysis consists of running the program and observing its behavior through a controlled environment such as a virtual machine [4]. However, this can be risky since some malware can detect the presence of a virtual machine and infect the host system or connect to the network to infect other systems. Conversely, static analysis examines the program code structure without executing it to learn its behavior, allowing the malware structure to be understood without risking system infection by executing the program [4].

One specific method of static analysis for malware is software reverse engineering, which reverses the compilation process to recover parts of the program's original structure for analysis [3]. This can be an effective method for determining how a malicious program works so measures can be taken to combat it. While tools exist to assist reverse engineers in analyzing programs and malware, they often produce large amounts of data that are tedious to parse and understand.

This paper introduces MAGE-ontology, an ontology for malware analysis in Ghidra for executables. MAGE-ontology organizes the data from Ghidra's analysis of an executable to identify and categorize malicious patterns of behavior. This ontology was evaluated through an ontology pitfall scanner and

ISWC'26: International Semantic Web Conference, October 25–29, 2026, Bari, Italy

✉ miller.1771@wright.edu (E. Miller); cogan.shimizu@wright.edu (C. Shimizu)

🌐 <https://cogan-shimizu.github.io/> (C. Shimizu)

🆔 0009-0008-4838-9461 (E. Miller); 0000-0003-4283-8701 (C. Shimizu)



© 2026 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

used to materialize knowledge graphs based on executable files. These knowledge graphs were tested through querying to detect common patterns of malware behavior.

This ontology is licensed under a Creative Commons Attribution 4.0 International License. All the associated code, resources, and other documentation for this ontology can be found in the following GitHub repository¹. The associated Ontology Design Pattern (ODP) for this ontology was accepted into the public ODP catalog, which can be found here²

2. Related Work

Various frameworks and ontologies have been created to identify malware, with static analysis techniques frequently utilized to structure and represent the underlying data. One such framework is the SEPSES knowledge graph, which integrates multiple cybersecurity databases and standards into a single graph [5]. This knowledge graph is consistently updated with the latest vulnerabilities from these databases, making it an up-to-date resource on the latest threats in a dynamically changing environment. While this provides a comprehensive framework for identifying known malware, attacks, and vulnerabilities, it does not have the functionality to detect novel attacks not present in these databases.

An agentic framework that extends the functionality of the SEPSES knowledge graph, AgCyRAG, using it as a SPARQL endpoint connecting it with multiple other agents [6]. It utilizes knowledge graphs with vector-based retrieval and an agentic design to obtain semantic context that better represents the complex environment of cybersecurity and allows for complex multi-step analysis based on user queries [6]. However, even with this increased functionality, it still lacks the ability to detect novel attacks and malware.

A better-suited technique for detecting novel malware is through code analysis, as malicious behaviors can be detected this way even if the specific attack or vulnerability has not been discovered [7]. CodeOntology is a framework that formally represents source code from object-oriented languages [8]. While designed to adapt to multiple languages, the framework focuses primarily on Java, utilizing a custom parser to convert source code into Resource Description Framework (RDF) triples that populate a knowledge graph. This is useful for understanding how source code can be represented formally, but it usually cannot be applied from a malware analysis standpoint, since malware source code is often unavailable. The ontology proposed in this research focuses on disassembled C code provided by Ghidra to analyze executable files without their source code.

3. Background

Software reverse engineering is the process of analyzing an executable file in such a way that the structure and functionality of a program can be understood without access to the source code [9]. Since malware's source code is usually unavailable, this process is one of the only ways to obtain parts of the original program structure. To expedite this process, reverse engineering tools are frequently employed; this research specifically utilizes Ghidra, an open-source software reverse engineering framework designed to assist analysts in examining executables and malware [10].

To analyze executable files, Ghidra reverses the compilation process to obtain fragments of what the source code could have looked like. The process of compilation consists of compiling high-level program code into assembly instructions, which are then assembled into machine code that can be executed by the system [11]. This process is lossy, since information such as variable and function names is not used by the machine when generating the executable and is therefore discarded to improve runtime efficiency [3]. Consequently, reconstructing the complete and exact source code from an executable file is virtually impossible, as the compilation process permanently discards critical metadata and structural information.

¹<https://github.com/kastle-lab/MAGE-Ontology>

²<https://github.com/odpa/patterns-repository>

While the original high-level code cannot be fully retrieved with 100% accuracy, Ghidra can still disassemble and decompile the executable to obtain at least some of that information back. The disassembly translates the machine code from a binary file into assembly instructions to grant human readability [12]. These instructions consist of operation codes (opcodes), which specify the operation on the data, and the operands, which are the actual data being modified through the instruction [3]. Since assembly instructions operate at a low level of abstraction in the system's memory, interpreting the program remains cognitively demanding. Ghidra also includes a decompilation of the executable file, which converts the assembly instructions into higher-level C code that can be parsed much more efficiently [10]. This decompilation is not entirely accurate to the source code because the compilation process is lossy, but it still provides useful insight into how the program works and is structured.

Ghidra also has other useful displays to understand the program's functionality. One of these is the symbol tree, which contains all the symbols in an executable [3]. A symbol is a link between an address and a string name, which allows data to have string labels to identify the data [13]. The symbol tree organizes these symbols based on the namespaces they are contained in. This is useful to assist reverse engineers in keeping track of variables, functions, and other types of data within the program to understand its functionality. Even with the useful tools provided by Ghidra and other reverse engineering frameworks, they often produce excess data about a file that can be difficult to track at scale. This is where the motivation to use an ontology to track this data originated.

4. Pattern Overview

The process of designing and developing the ontology followed the Modular Ontology Modeling methodology outlined in [14], which consisted of the following steps: developing competency questions, choosing the data set, drafting the schema, specifying the axioms, and writing the ontology. These steps, which outline the pattern overview, will be explained in detail in the following sections.

4.1. Competency Questions

The ontology was designed based on the competency questions listed below, which were derived from the use case of detecting patterns of malicious behavior in executables. Some of these patterns that are found in the assembly and decompilation include suspicious API calls [15] [16], self-referential decode loops [17], substantial entropy [18], cryptographic activity [2], and high percentages of imported files [19]. These questions were later used to develop the SPARQL queries for the knowledge graphs.

1. Can the ontology detect malicious persistent API calls within the executable?
2. Is there any suspicious networking activity within the executable?
3. Are there any indirect control flow patterns in the executable file indicating vulnerable code?
4. Can sections of entropy be detected through the ratio of unique opcodes to the total number of instructions in a function?
5. Can patterns of self-referential decode loops be found to detect data obfuscation?
6. Can the ontology detect file system abuse?
7. Are there any suspicious jumps into memory in the executable?
8. Are there any other suspicious API calls contained in the executable file?
9. Is there cryptographic activity within the executable that can indicate encryption to hide parts of the program?
10. What percentage of the functions in the binary file are imported (i.e., external)?

4.2. Selecting the Dataset

The dataset for the ontology was selected during the initial development of the competency questions. Since the use case for this ontology is to detect malicious behavior, an open-source GitHub repository containing both benign and malicious executables was chosen [20]. The detailed labels on the files proved

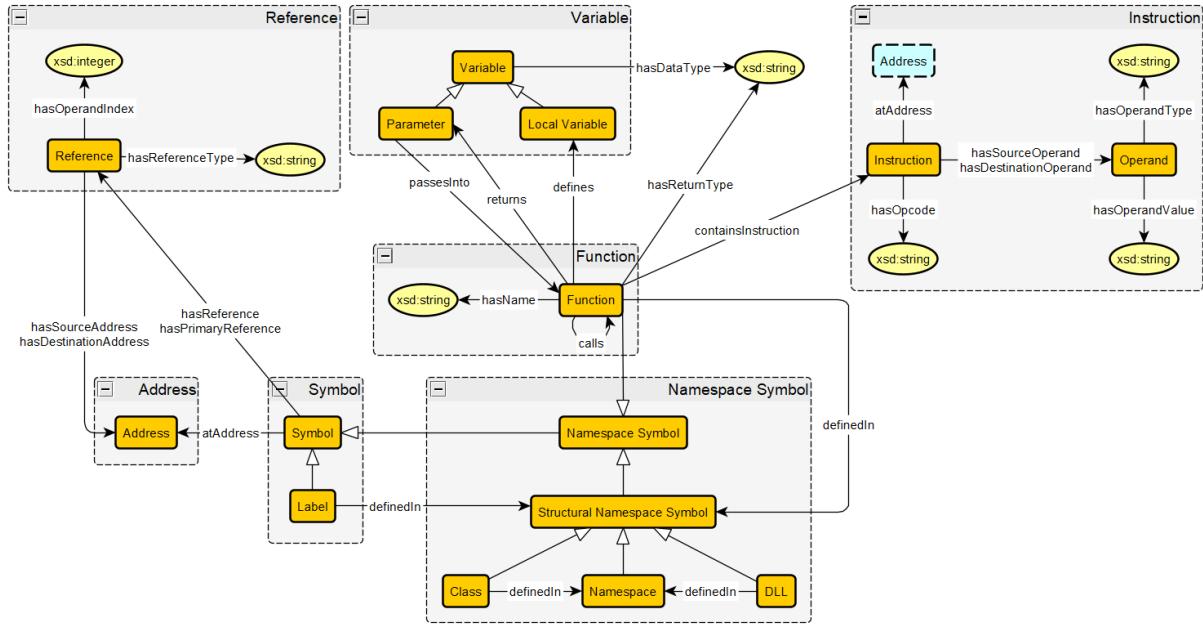


Figure 1: Schema of the ontology based on the symbol tree in Ghidra. Created using yEd graph editor. [21]

useful in understanding the expected behavior, allowing for behavior comparison in different executables. This repository contains simple files with detailed labels, which proved useful in understanding the category of malicious behavior they contain. These files are in the Executable and Linkable Format (ELF), the format Linux and other Unix-based systems use for executables. This posed a restriction on how the queries would be developed later, as API names differ between executable formats.

The ontology was designed around the symbol tree of an analyzed file from Ghidra. The symbol tree contains all the symbols of an executable file, including the imports, exports, functions, labels, classes, and namespaces [3]. Designing the ontology in this manner helps visualize patterns of how the symbols interact, allowing suspicious behavior to be detected.

4.3. Designing the Schema

Before the ontology was developed, a schema diagram was created using the software yEd to visually show how all the objects would relate to each other [21]. The schema includes objects for all symbol types (with imports and exports being encompassed by the Dynamically Linked Library (DLL) symbol), as well as other objects that help explain the functionality of the symbols. Other objects include types of variables found in functions, references that show where a symbol is referenced in memory, and assembly instructions that show the assembly code within a function. Figure 1 shows the final version of the schema from which the ontology was derived.

4.4. Formalization

Alongside designing the schema (as in the diagrammatic representation), we also formalized it, so as to further provide a more rigorous underpinning. Generally speaking, these axioms are derived from the 17 axiom patterns detailed in both [14] and [22].

Symbol

- | | | |
|--|-----------------|-----|
| $\text{Symbol} \sqsubseteq \forall \text{ hasReference.Reference}$ | (Scoped Range) | (1) |
| $\text{Symbol} \sqsubseteq \leq 1 \text{ hasPrimaryReference.Reference}$ | (Functionality) | (2) |
| $\text{Symbol} \sqsubseteq \leq 1 \text{ hasAddress.Address}$ | (Functionality) | (3) |

Label $\sqsubseteq$ Symbol	(Subclass)	(4)
NamespaceSymbol $\sqsubseteq$ Symbol	(Subclass)	(5)
		(6)

A symbol is a named entity in an executable file that is associated with a specific memory address. This simplifies the reverse engineering process by giving human-readable names to data. In this schema, the two main subclasses of symbol are labels and namespace symbols, with the other kinds of symbols being subclasses of the namespace symbol.

Namespace Symbol

StructuralNamespaceSymbol $\sqsubseteq$ NamespaceSymbol	(Subclass)	(7)
Namespace $\sqsubseteq$ StructuralNamespaceSymbol	(Subclass)	(8)
DLL $\sqsubseteq$ StructuralNamespaceSymbol	(Subclass)	(9)
Class $\sqsubseteq$ StructuralNamespaceSymbol	(Subclass)	(10)
Label $\sqsubseteq \leq 1$ definedIn.StructuralNamespaceSymbol	(Functionality)	(11)
Class $\sqsubseteq \leq 1$ definedInNamespace.Namespace	(Functionality)	(12)
DLL $\sqsubseteq \leq 1$ definedInNamespace.Namespace	(Functionality)	(13)

A namespace symbol is a kind of symbol in Ghidra that is used to organize other kinds of symbols. A function is a type of namespace symbol since it holds local variables. A further distinction is made: structural namespace symbols, which can hold more global namespaces and objects, like functions and labels. This means that classes, DLLs, and namespaces can all hold functions and labels. Ghidra does not support nested classes and nested functions in its decompilation, so those aspects are not represented in this schema. Classes and DLLs are also defined within a namespace, with all DLLs defined in the global namespace.

Reference

Reference $\sqsubseteq \leq 1$ hasSourceAddress.Address	(Functionality)	(14)
Reference $\sqsubseteq \leq 1$ hasDestinationAddress.Address	(Functionality)	(15)
Reference $\sqsubseteq \leq 1$ hasReferenceType.xsd:string	(Functionality)	(16)
Reference $\sqsubseteq \leq 1$ hasOperandIndex.xsd:integer	(Functionality)	(17)

A reference is where two memory addresses interact with each other in some way, where one address uses another. This is used for things like when a function calls another function or when data is accessed by an instruction. References are 4-tuples that include the source address, the destination address, the type of reference (function call, data being accessed, etc.), and the operand index (which is an int that is either -1, 0, or 1).

Function

Function $\sqsubseteq$ NamespaceSymbol	(Subclass)	(18)
Function $\sqsubseteq \forall$ definesLocal.LocalVariable	(Scoped Range)	(19)
Function $\sqsubseteq \leq 1$ hasReturnType.xsd:string	(Functionality)	(20)
Function $\sqsubseteq \leq 1$ returns.Parameter	(Functionality)	(21)
Function $\sqsubseteq \forall$ calls.Function	(Scoped Range)	(22)
Function $\sqsubseteq \leq 1$ definedIn.StructuralNamespaceSymbol	(Functionality)	(23)

Function $\sqsubseteq \forall$ containsInstruction.Instruction	(Scoped Range)	(24)
Function $\sqsubseteq \leq 1$ hasName.xsd:string	(Functionality)	(25)

The function object keeps track of all the aspects of a function, including any functions it calls or functions called by it, the parameters passed in, the local variables defined in the function, the return type of the function, the return parameter of the function, the instructions the function contains, and what class or namespace the function is contained in.

Variable

Parameter $\sqsubseteq$ Variable	(Subclass)	(26)
LocalVariable $\sqsubseteq$ Variable	(Subclass)	(27)
Variable $\sqsubseteq \forall$ hasDataType.xsd:string	(Scoped Range)	(28)
Parameter $\sqsubseteq \forall$ passesInto.Function	(Scoped Range)	(29)

Variables in this context are tied directly to functions, and not a type of symbol itself. There are two types of variables in this schema: parameters, which are passed into and returned from functions, and local variables, which are defined locally within the function itself. Every variable has a data type associated with it.

Instruction

Instruction $\sqsubseteq \leq 1$ hasOpcode.xsd:string	(Functionality)	(30)
Instruction $\sqsubseteq \forall$ hasSourceOperand.Operand	(Scoped Range)	(31)
Instruction $\sqsubseteq \leq 1$ hasDestinationOperand.Operand	(Functionality)	(32)
Instruction $\sqsubseteq \leq 1$ atAddress.Address	(Functionality)	(33)
Operand $\sqsubseteq \leq 1$ hasOperandType.xsd:string	(Functionality)	(34)
Operand $\sqsubseteq \leq 1$ hasOperandValue.xsd:string	(Functionality)	(35)

The instruction object refers to an assembly instruction that originates from the disassembly obtained by Ghidra from an executable. An instruction includes one opcode and zero or more operands. The opcode is represented as a string, while the operand is represented as an object with two values: a string representing the operand type and a string representing the operand value.

4.5. Creating the Ontology

Once the schema and axiomization were specified, the ontology was developed through the ontology editor Protégé [23]. This tool allows classes, object properties (relations between objects), and data properties (relations between an object and a literal data value) to be specified. The tool also allows for other attributes to be specified, such as the domain and range of certain relations (the types of objects that can use the relation), whether a relation is functional, labels and comments for objects, and more. The ontology can be saved to multiple RDF formats, with the Turtle format used for this research. The saved Turtle file became the basis of the materialized knowledge graphs, which can be found here³.

5. Evaluation

This section provides an overview of the evaluation of the ontology through an ontology pitfall scanner and the application through knowledge graph materialization and SPARQL query generation.

³<https://github.com/kastle-lab/MAGE-Ontology/tree/6cbf0e61a573a53db6d426329a314aa4ade1f03c/ontology>

5.1. Ontology Evaluation

The ontology was evaluated through the OOPS! Ontology Pitfall Scanner tool [24]. This tool detects common errors in ontologies, ranging from minor to severe, and provides the object or relation in which the error resides. Multiple issues were fixed by this tool, including no definitions for explicit inverse relations, missing domains and ranges, and objects with no outgoing relations. This tool was useful for both an initial check of the ontology and as a check after modifications were made during the process. Initially, two moderate errors were detected but not changed. The first is a recursive definition for the function object. This happens in the ‘function calls function’ relation. Since it is important to visualize function calls, this relation was not removed. The other error was for equivalent classes not explicitly declared for the address and reference objects. This error was detected due to the names used for address and reference (which often refer to the same thing in a similar context), but since in the ontology they are defined as two separate objects, this error was disregarded.

5.2. Knowledge Graph Materialization

Multiple knowledge graphs were materialized from the ontology, and multiple executable files (one per knowledge graph) were analyzed by Ghidra. The process of this materialization consisted of extracting data from the dataset through API calls, parsing and organizing the data, and then storing it as RDF triples that constitute the knowledge graph.

5.2.1. Data Extraction and Parsing

The data to be used for the knowledge graph is specifically the data extracted from a file that Ghidra analyzed. Once a new Ghidra project is started and the executable file is imported, Ghidra will analyze the file, providing the disassembly, decompilation, symbol tree, and other views about the file. Then, Ghidra’s script manager can be utilized to run Java or Python programs directly through Ghidra on the analyzed executable. A data extraction program was created in Python using pyGhidra, which is the library that allows Python scripts to be executed from Ghidra. This script utilized Ghidra API calls to gather data about the file [13].

The script produced multiple data files for the different objects and data from the executable. A script was written to parse these data files and convert the data into Python dictionaries, which would then be iterated over during the creation of the triples in the materialization step. Each object has its own dictionary, with the key-value pairs indicating the attributes for that object. For example, a variable object contained key-value pairs for the variable name, its data type, and the parent function in which it is defined. These dictionaries were then inserted into lists that contained all the dictionaries of that object type, which were then iterated over to create the RDF triples in the materialization step.

5.2.2. Materialization

Once the data was organized into these dictionaries, a materialization script was written to format the data into RDF triples and add these triples to a new knowledge graph. This script was built on top of the RDFLib starter code provided by the Kastle Drawbridge GitHub Repository, which included the imports needed, as well as initialized and serialized a test graph [25]. The script first parsed the ontology file and added all the structural triples that define how the data triples will be constructed. Then, the script iterated through all the lists of dictionaries for all object types, created the RDF IRIs for each object, and then used the properties stored in the dictionaries to create the triples for all the relations that exist for that object. After all the triples are created, the graph is serialized and stored in another Turtle file representing the completed knowledge graph.

5.3. SPARQL Queries

SPARQL queries were developed from the competency questions to detect malicious behavior within the executable. Many of these queries searched for specific API call names that are unique to ELF binary

files, like the query in Figure 3. This means that different queries would need to be made to analyze other formats of executable files. Other queries look for specific types of assembly instructions within a given function, where some instructions in a certain order could indicate specific types of malicious behavior. These queries were run through the SQL-server Apache Jena Fuseki to obtain results from the knowledge graph [26].

```
SELECT ?f ?fname ?name
WHERE {

    ?f ontology:calls ?api .
    ?f ontology:hasName ?fname .
    ?api ontology:hasName ?name .

    FILTER regex(?name,
        "socket|connect|send|recv|sendto|recvfrom",
        "i")
}
```

Figure 2: SPARQL query based on 2nd competency question: "Is there any suspicious networking activity within the executable?"

Thirteen queries were created in total, one for each competency question, with question nine having three queries. This paper will examine a few queries in more detail; all the queries and their related competency question can be found in the documentation section of the project repository [27]. The query in Figure 5.3 searches for specific networking API calls, such as "socket" and "connect," which can indicate suspicious networking activity within the executable, especially when detected in higher amounts.

The query in Figure 5.3 is based on the fourth competency question, which finds the ratio between the number of unique opcodes and the total number of instructions found in the functions of the file. The query will list this entropy ratio for functions that contain more than ten instructions. A higher ratio indicates there are more unique opcodes compared to total instructions, which is a sign of entropy within the function. High levels of entropy often indicate that the code is either encrypted or obfuscated in some way, which is a tactic malware uses to conceal its true purpose. Detecting functions with high entropy, therefore, indicates a higher likelihood that the executable is malicious.

```
SELECT ?function
    (COUNT(DISTINCT ?opcode) AS ?uniqueOpcodes)
    (COUNT(?instruction) AS ?totalInstructions)
    ((COUNT(DISTINCT ?opcode) * 1.0 / COUNT(?instruction))
    AS ?entropyRatio)

WHERE {
    ?function ontology:containsInstruction ?instruction .
    ?instruction ontology:hasOpcode ?opcode .
}
GROUP BY ?function
HAVING (?totalInstructions > 10)
ORDER BY DESC(?entropyRatio)
```

Figure 3: SPARQL query based on 4th competency question: "Can sections of entropy be detected through the ratio of unique opcodes and total number of instructions in a function?"

File	# of Networking API Calls
BASHLITE Client	32
Lizkebab Server	11
Mirai Loader	17

Table 1

Query results based on competency question 2: "Is there any suspicious networking activity within the executable?"

File	Function	Unique Opcodes	# of Instructions	Entropy Ratio
BASHLITE Client and Mirai Loader	register_tm_clones	9	12	0.75
BASHLITE Client and Mirai Loader	deregister_tm_clones	10	14	0.7143
Lizkebab Server	register_tm_clones	9	14	0.6429
BASHLITE Client and Mirai Loader	register_tm_clones	11	18	0.6111

Table 2

Query results based on competency question 4: "Can sections of entropy be detected through the ratio of unique opcodes and total number of instructions in a function?" Showing all functions found in the files that have a ratio of 0.6 or higher.

5.4. Analysis

A total of three knowledge graphs were materialized based on three malicious ELF files chosen from the dataset [20]. The first file is a client component from the BASHLITE malware family, which is a botnet that conducts denial-of-service attacks on IoT devices [28]. The second file is a Lizkebab server, where Lizkebab is a variant of the BASHLITE malware family. The final file is a Mirai-vanilla loader, where Mirai is the malware family that succeeded the BASHLITE family [28]. A set of 10 SPARQL queries was run on each of these knowledge graphs to showcase the results based on the different graphs. All the queries and their results can be found in the documentation section of the project GitHub repository [27].

Table 2 showcases the number of networking API calls detected in each executable based on the query in Figure 2. Networking APIs indicate that the file connects to the network in some way, which can indicate malicious interactions between bots and the malicious server. The query successfully captures the networking activity of the types of malware, with the client executable having the most networking calls at 32. Client-side bots make frequent connections to the malicious server to either receive instructions on what to do or send data back over to the server. This interaction is showcased through the client BASHLITE executable, containing noticeably more networking API calls compared to the other files.

Table 3 shows the results of the query based on competency question 4, which calculated an entropy ratio for the functions contained within the executables. The results listed in the table are all the functions found that had an entropy ratio greater than 0.6. The number 0.6 was chosen to showcase the most notable examples of entropy detected within the data. Some functions are contained in both the BASHLITE client and the Mirai Loader, indicating that they are most likely imported functions used in multiple kinds of malicious executables. These shared functions have noticeably higher entropy ratios compared to the other functions, indicating they could be contributing to either encryption, data obfuscation, or other functionality used for malicious intent.

Some queries only obtained results for particular executables. For example, competency question 9 was broken down into 3 separate queries. The first query detected functions with at least ten XOR operations, with large amounts of XOR operations indicating custom encryption algorithms. The second query was to identify any API calls to encryption algorithms. The final query detected patterns through

File	Percentage of Functions Imported
BASHLITE Client	52.29%
Lizkebab Server	68.65%
Mirai Loader	51.92%

Table 3

Query results based on competency question 10: "What percentage of the functions in the binary file are imported (are external)?"

bitwise operations in loops within functions that could also indicate custom encryption algorithms. Out of all these queries, only the BASHLITE Client file returned relevant results. While it did not detect any API calls to defined encryption algorithms, it found both instances of a large number of XOR operations and other bitwise operations within certain functions. This could indicate that only the client side of these kinds of malware would contain custom encryption routines. Since the client bot is the program located on the victim's system, having the data encrypted is more important for the attacker, so the victim cannot get access to the contents of the executable or the network communication between the bot and the server.

Table 4 contains the percentages of imported functions for each executable, based on competency question 10. A higher percentage of imported functions could indicate that the functions are defined elsewhere to obscure their functionality, making it more difficult to identify where the malicious behavior is originating from. There are also known libraries that contain malicious functions that are used in malware, so taking a closer look could reveal that these functions are known to be malicious.

6. Conclusion

In this paper, we introduced MAGE-ontology: a domain ontology that models the data of an executable file analyzed by Ghidra to detect malicious behavior at the assembly level. The ontology was evaluated using the OOPS! Ontology Pitfall Scanner to fix common issues and standardize its structure. The ontology was tested by materializing multiple knowledge graphs from malicious executable files, and queries were created based on the competency questions to evaluate the usefulness of the graphs. The queries detected different forms of malicious behavior, including suspicious networking activity, extensive use of imported functions, and high levels of entropy within functions.

6.1. Future Work

This ontology was primarily designed to work with Ghidra, but it has the potential to assist with different reverse engineering tools. Minor changes in the way the symbols and data are represented could be made to make it more applicable to other reverse engineering tools while keeping the same fundamentals. This could allow the ontology to be more flexible and more widely used in reverse engineering based on tool and framework preferences. The ontology could also be integrated into a Ghidra plugin to allow a knowledge graph to be generated directly within Ghidra for a given executable file. This could make the ontology more accessible for direct analysis of a binary within the application. Plugins could also be made for other software to allow direct knowledge graph materialization through multiple tools.

Acknowledgments. Cogan Shimizu acknowledges support from NSF Awards #2537485, #2514820, and #2333532.

Declaration on Generative AI. A combination of Gemini 3 Flash and GPT-5.3 Instant was used to help generate and modify the SPARQL queries to fit the competency questions. Grammarly was used for grammar and spell checking for the preparation of this work. The authors reviewed and edited the content afterward and take full responsibility for the content of the publication.

References

- [1] J. Ferdous, R. Islam, A. Mahboubi, M. Islam, A Review of State-of-the-Art Malware Attack Trends and Defense Mechanisms, *IEEE Access* 11 (2023). doi:10.1109/ACCESS.2023.3328351.
- [2] P. McLaren, W. J. Buchanan, G. Russell, Z. Tan, Discovering encrypted bot and ransomware payloads through memory inspection without a priori knowledge, 2019. URL: <https://arxiv.org/abs/1907.11954>. arXiv:1907.11954.
- [3] K. Nance, C. Eagle, *The Ghidra Book: The Definitive Guide*, No Starch Press, 2020.
- [4] E. Gandotra, D. Bansal, S. Sofat, Malware analysis and classification: A survey, *Journal of Information Security* 5 (2014) 56. doi:10.4236/jis.2014.52006.
- [5] E. Kiesling, A. Ekelhart, K. Kurniawan, F. Ekaputra, The sepses knowledge graph: An integrated resource for cybersecurity, in: C. Ghidini, O. Hartig, M. Maleshkova, V. Svátek, I. Cruz, A. Hogan, J. Song, M. Lefrançois, F. Gandon (Eds.), *The Semantic Web – ISWC 2019*, Springer International Publishing, Cham, 2019, pp. 198–214.
- [6] K. Kurniawan, R. F. Ardian, E. Kiesling, A. Ekelhart, Agcyrag: an agentic knowledge graph based rag framework for automated security analysis, in: *RAGE-KG@ISWC*, 2025. URL: <https://api.semanticscholar.org/CorpusID:282764790>.
- [7] J. J. Sara, S. Hossain, Static analysis based malware detection for zero-day attacks in android applications, in: *2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD)*, 2023, pp. 169–173. doi:10.1109/ICICT4SD59951.2023.10303336.
- [8] M. Atzeni, M. Atzori, Codeontology: Rdf-ization of source code, in: C. d’Amato, M. Fernandez, V. Tamma, F. Lecue, P. Cudré-Mauroux, J. Sequeda, C. Lange, J. Heflin (Eds.), *The Semantic Web – ISWC 2017*, Springer International Publishing, Cham, 2017, pp. 20–28.
- [9] S.-Y. Yu, Y. G. Achamyeh, C. Wang, A. Kocheturov, P. Eisen, M. A. A. Faruque, Cfg2vec: Hierarchical graph neural network for cross-architectural software reverse engineering, 2023. URL: <https://arxiv.org/abs/2301.02723>. arXiv:2301.02723.
- [10] D. Votipka, M. N. Punzalan, S. M. Rabin, Y. Tausczik, M. L. Mazurek, An investigation of online reverse engineering community discussions in the context of ghidra, in: *2021 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2021, pp. 1–20. doi:10.1109/EuroSP51992.2021.00012.
- [11] A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman, *Compilers Principles, Techniques, & Tools*, Addison Wesley, 2007. Edition: 2nd.
- [12] M. Sikorski, A. Honig, *Practical Malware Analysis: The Hands-On Guide to Dissecting Malicious Software*, No Starch Press, San Francisco, CA, 2012.
- [13] N. S. Agency, Interface symboltable - ghidra api documentation, https://ghidra.re/ghidra_docs/api/ghidra/program/model/symbol/SymbolTable.html, 2026.
- [14] C. Shimizu, K. Hammar, P. Hitzler, Modular ontology modeling, *Semantic Web* 14 (2023) 459–489. URL: <https://doi.org/10.3233/SW-222886>. doi:10.3233/SW-222886.
- [15] T. Tam, D. Tran, N. Trong, N. Tuan, et al., Malware detection using assembly and API call sequences, *International Journal of Information Security* 9 (2010) 319–331. URL: <https://doi.org/doi:10.1007/s11416-010-0141-5>.
- [16] M. Alazab, S. Venkataraman, P. Watters, Towards understanding malware behaviour by the extraction of api calls, in: *2010 Second Cybercrime and Trustworthy Computing Workshop*, 2010, pp. 52–59. doi:10.1109/CTC.2010.8.
- [17] D. Bruschi, L. Martignoni, M. Monga, Detecting self-mutating malware using control-flow graph matching, in: R. Büschkes, P. Laskov (Eds.), *Detection of Intrusions and Malware & Vulnerability Assessment*, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 129–143.
- [18] K. Vahedi, M. Abbaspour, K. Afhamisizi, M. Rashidnejad, Behavioral entropy towards detection of metamorphic malwares, in: *2019 9th International Conference on Computer and Knowledge Engineering (ICCKE)*, 2019, pp. 78–84. doi:10.1109/ICCKE48569.2019.8964967.
- [19] T. Lu, X. Liu, J. Chen, N. Hu, B. Liu, Afcgdroid: Deep learning based android malware detec-

- tion using attributed function call graphs, *Journal of Physics: Conference Series* 1693 (2020) 012080. URL: <https://iopscience.iop.org/article/10.1088/1742-6596/1693/1/012080>. doi:10.1088/1742-6596/1693/1/012080.
- [20] N. Partush, Labeled-elfs, <https://github.com/nimrodpar/Labeled-Elfs>, 2024. Accessed on: 12 March 2026.
 - [21] yWorks GmbH, yed, 2019. URL: <https://www.yworks.com/products/yed>.
 - [22] A. Eberhart, C. Shimizu, S. Chowdhury, M. K. Sarker, P. Hitzler, Expressibility of OWL axioms with patterns, in: R. Verborgh, K. Hose, H. Paulheim, P. Champin, M. Maleshkova, Ó. Corcho, P. Ristoski, M. Alam (Eds.), *The Semantic Web - 18th International Conference, ESWC 2021, Virtual Event, June 6-10, 2021, Proceedings*, volume 12731 of *Lecture Notes in Computer Science*, Springer, 2021, pp. 230–245. URL: https://doi.org/10.1007/978-3-030-77385-4_14. doi:10.1007/978-3-030-77385-4_14.
 - [23] M. A. Musen, The protégé project: a look back and a look forward, *AI Matters* 1 (2015) 4–12. URL: <https://doi.org/10.1145/2757001.2757003>. doi:10.1145/2757001.2757003.
 - [24] M. Poveda-Villalón, A. Gómez-Pérez, M. C. Suárez-Figueroa, OOPS! (OntOlogy Pitfall Scanner!): An On-line Tool for Ontology Evaluation, *International Journal on Semantic Web and Information Systems (IJSWIS)* 10 (2014) 7–34.
 - [25] Kastle-Lab, Kastle-Drawbridge, GitHub, 2026. URL: <https://github.com/kastle-lab/kastle-drawbridge>, accessed: 2026-03-20.
 - [26] Apache Software Foundation, Apache jena fuseki, <https://jena.apache.org/documentation/fuseki2/>, 2026. Accessed: 2026-03-20.
 - [27] E. Miller, MAGE-Ontology, <https://github.com/kastle-lab/MAGE-Ontology>, 2026.
 - [28] A. Marzano, D. Alexander, O. Fonseca, E. Fazzion, C. Hoepers, K. Steding-Jessen, M. H. P. C. Chaves, Í. Cunha, D. Guedes, W. Meira, The evolution of bashlite and mirai iot botnets, in: *2018 IEEE Symposium on Computers and Communications (ISCC)*, 2018, pp. 00813–00818. doi:10.1109/ISCC.2018.8538636.